



Nädal 7

Nädal 7: Python ja Pandas

SQL on alus. Python on supervõime. — Marko Saare Väljakutse

Marko Saar, Product Manager

Sessiooni ülevaade (90 min)

Faas	Aeg	Sisu
CONNECTION	0:00–0:15	Marko Saare videosõnum, SQL vs Python haak
CONCEPTS	0:15–0:45	Python keskkond, pandas DataFrame anatoomia, RFM raamistik
CONCRETE PRACTICE	0:45–1:20	Live kodeerimine, andmete laadimine ja uurimine, harjutused
CONCLUSIONS	1:20–1:30	Refleksioon, Sessioon 2 eelvaade: RFM segmenteerimine



CONNECTION

0:00–0:15

Marko Saare väljakutse ja SQL → Python sild

Marko Saare sõnum

Marko Saar, Product Manager

"Hey team! Ma vajan abi. Dashboard on cool, aga ma vajan SEGMENTE. Ma tahan saata erinevaid e-maile VIP-dele vs newbies'tele. Python on key? Õpetage mulle (ja endale)!"



Jäämurdja — Python'i kogemus

- ◆ Kes on Python'iga varem kokku puutunud? Käsi üles!
- ◆ Kes on täielik algaja? — Ärge muretsege!
- ◆ SQL-ist tulek on tegelikult väga hea ettevalmistus
- ◆ Kõik, mida te SQL-is teete, saab teha ka Python'is — ja PALJU enamat



SQL vs Python — miks Python?

SQL (mida te juba oskate)

- ◆ `SELECT customer_id, COUNT(*) FROM sales GROUP BY customer_id`
- ◆ Annab tabeli — ja siis?
- ◆ Keeruline arvutusi teha
- ◆ Eksportimine piiratud

Python + pandas

- ◆ `df.groupby('customer_id')['total_amount'].sum()`
- ◆ Saad KOHE edasi töödelda!
- ◆ Kvintilid, skoorid, segmendid
- ◆ CSV eksport, visualisatsioon, automatiseerimine



CONCEPTS

0:15–0:45

Python keskkond, DataFrame anatoomia, RFM raamistik

Python keskkonna kontroll

```
# Terminalis:  
# python --version          → Peaks näitama 3.11+  
# pip install pandas jupyter supabase-py plotly python-dotenv  
  
# Python'is:  
import pandas as pd  
print(pd.__version__)      # pandas versiooni kontroll  
  
# Kui ei tööta → varuplaanina kasuta Google Colab'i
```

Kontrolli, et Python 3.11+ ja pandas on installeeritud. Jupyter Notebook on meie peamine töökeskkond.

Python vs SQL analoogia — mõtle SQL-is, kirjuta Python'is

SQL	Python + pandas	Selgitus
SELECT * FROM sales LIMIT 5	df.head()	Esimesed 5 rida
SELECT COUNT(*) FROM sales	len(df)	Ridade arv
WHERE amount > 100	df[df['amount'] > 100]	Filtreerimine
GROUP BY city	df.groupby('city')	Grupeerimine
JOIN ... ON	pd.merge(df1, df2, on='key')	Tabelite ühendamine
ORDER BY total DESC	df.sort_values('total', ascending=False)	Sorteerimine



DataFrame'i anatoomia

- ◆ DataFrame = SQL tabel Python'is — READ ja VEERUD
- ◆ Iga veerg on Series (üks andmetüüp)
- ◆ Index = reanumber (nagu SQL PRIMARY KEY)
- ◆ dtypes: int64, float64, object (tekst), datetime64
- ◆ shape: (ridade arv, veergude arv)

pandas DataFrame struktuur

`df.index`

`df.columns (4 tk)`

`df.shape = (5, 4)`

Indeks	name	city	amount	date
0	Anna	Tallinn	129.90	2024-01-15
1	Marko	Tartu	89.50	2024-02-20
2	Liis	Parnu	210.00	2024-03-10
...	...			
4	Kristi	Tallinn	55.00	2024-05-01

df.dtypes:
name: object | city: object
amount: float64 | date: datetime64

Kasulikud meetodid:
df.head() df.info() df.describe()
df.shape df.columns df.dtypes



DataFrame'i tutvumine — esimesed käsud

```
import pandas as pd

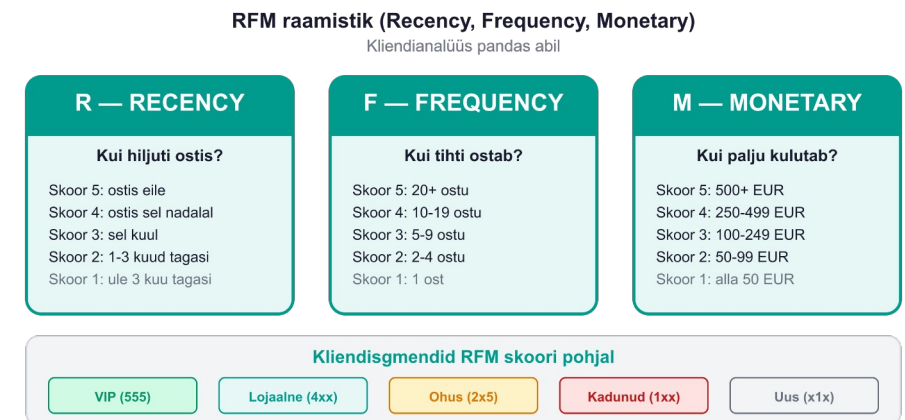
# Laadi andmed CSV-st (või Supabase'ist)
df = pd.read_csv('sales.csv')

# Tutvumine — 4 põhikäsku
print("Shape:", df.shape)          # (10118, 12) — read × veerud
print(df.head())                   # Esimesed 5 rida (nagu SELECT * LIMIT 5)
print(df.info())                   # Veerutüübid, NULL-ide arv
print(df.describe())               # Statistiline kokkuvõte (MIN, MAX, AVG)
```

head() = SELECT TOP 5, info() = \d (PostgreSQL tabelikirjeldus), describe() = statistika. Neli käsku, millega alustada ALATI.

RFM raamistik — kliendisegmenteerimine

- ◆ R = Recency: Kui HILJUTI klient ostis? (R=5 → ostis hiljuti)
- ◆ F = Frequency: Kui TIHTI klient ostab? (F=5 → 10+ ostu)
- ◆ M = Monetary: Kui PALJU klient kulutab? (M=5 → EUR500+)
- ◆ RFM Score = R + F + M (vahemik 3–15)
- ◆ Kvintiiipõhine: iga 20% klientidest saab skoori 1–5



RFM segmendid — mida skoor tähendab?

RFM Score	Segment	Kirjeldus
13–15	VIP Champions	Ostab tihti, hiljuti ja palju — kõige väärtuslikumad
10–12	Loyal Customers	Regulaarsed ja usaldusväärsed kliendid
7–9	Regular / Potential	Keskmised — potentsiaal kasvuks
4–6	At Risk	Varem aktiivsed, nüüd kadumas — kohene tegevus!
3	Lost	Kaua tagasi, harva, vähe — kas tuleb tagasi?



Miks RFM? — Marko ärivajadus

- ◆ VIP-dele: Early access + sooduskoodid → hoidmine
- ◆ At-Risk klientidele: 'Me igatseme sind!' kampaania → tagasivõitmine
- ◆ Uutele klientidele: Onboarding programm → kasvatamine
- ◆ ILMA segmenteerimiseta: kõik saavad SAMA e-maili → madal konversioon
- ◆ SQL ei suuda kvintilide automaatset arvutust — Python saab!



CONCRETE PRACTICE

0:45–1:20

Reaalajas kodeerimine Jupyter Notebook'is

Samm 1: Andmete laadimine

```
# Variant A: Supabase'ist
from supabase import create_client
import pandas as pd
import os

supabase = create_client(os.getenv("SUPABASE_URL"), os.getenv("SUPABASE_KEY"))
response = supabase.table('sales').select('*').execute()
df = pd.DataFrame(response.data)

# Variant B: CSV-st (varuplaanina)
# df = pd.read_csv('sales.csv')

print("Dataset shape:", df.shape)
print(df.head())
```

Kaks laadimisviisi: Supabase API (reaalajas) või CSV fail (varuplaanina). DataFrame on alati sama struktuuriga.

Samm 2: Põhiline uurimine

```
# Põhistatistika – nagu SQL SELECT COUNT, SUM, AVG
print("Total Revenue:", df['total_amount'].sum())
print("Average Order:", df['total_amount'].mean())
print("Number of Customers:", df['customer_id'].nunique())

# Grupeerimine – nagu SQL GROUP BY
city_revenue = df.groupby('city')['total_amount'].sum()
print("Revenue by City:")
print(city_revenue)
```

Tuttavad SQL operatsioonid pandas süntaksis. $sum() = SUM()$, $mean() = AVG()$, $nunique() = COUNT(DISTINCT)$.

Välkharjutus 1 — pandas Põhitõed

Kirjutage Jupyter Notebook'is järgmised pandas käsud:

```
# 1. Leia TOP 5 toodet kogukäibe järgi
top_products = df.groupby('product_name')['total_amount'].sum().nlargest(5)
print(top_products)

# 2. Leia klient suurima kogukulutusega
top_customer = df.groupby('customer_id')['total_amount'].sum().idxmax()
print("Top customer:", top_customer)
```

💡 Vihje: Kasuta `groupby()` + `sum()` + `nlargest(5)`. Mõtle SQL-i analoogias: `GROUP BY + ORDER BY DESC LIMIT 5`



Välkharjutus 2 — Filtreerimine ja sorteerimine

Edasijõudnute lisaülesanne: rakendage filter ja sortimine.

```
# 1. Filtreeri: ainult 2024. aasta andmed
df['order_date'] = pd.to_datetime(df['order_date'])
df_2024 = df[df['order_date'].dt.year == 2024]
print("2024 ridade arv:", len(df_2024))

# 2. Sorteeri: suurimad tellimused
df.sort_values('total_amount', ascending=False).head(10)
```

💡 Vihje: `pd.to_datetime()` teisendab teksti kuupäevaks. `.dt.year` annab aasta.



CONCLUSIONS

1:20–1:30

Refleksioon ja Sessioon 2 eelvaade

Kokkuvõte — mida me täna õppisime?

- ◆ Python + pandas = SQL-i laiendus, mitte asendus
- ◆ DataFrame = SQL tabel Python'is (read × veerud)
- ◆ head(), info(), describe(), shape — 4 põhikäsku
- ◆ groupby() = GROUP BY, merge() = JOIN, sort_values() = ORDER BY
- ◆ RFM raamistik: Recency, Frequency, Monetary — kliendisegmenteerimine



Kokkuvõte — mida me täna õppisime? (jätkub)

- ◆ Kvintiilipõhine skoor (1–5) annab suhtelise hinnangu



Refleksioon

Küsimused aruteluks

- ◆ Python vs SQL — milline tundub lihtsam?
- ◆ Milline tundub keerulisem? Miks?
- ◆ Mis on DataFrame ja SQL tabeli vaheline erinevus?
- ◆ Kus sa näed Python'i eelist SQL-i ees?

Ha-tase meeldetuletus

- ◆ Te juba TUNNETE andmeanalüüsi SQL-ist
- ◆ Python on UUS SÜNTAKS, mitte uus mõtlemine
- ◆ Iga süntaksiviga on õppimisvõimalus
- ◆ Veateade on teie sõber, mitte vaenlane!



Eelvaade: Sessioon 2 — RFM Kliendisegmenteerimine

Marko Saar, Product Manager

"Homme teeme RFM analüüsi! Me segmenteerime UrbanStyle.ltd kliente ja anname Markole konkreetsed soovitused. Iga meeskonnaliige vastutab ühe analüüsietapi eest: Data Loading, Cleaning, RFM Analysis, Visualization."



Järgmised sammud

Ettevalmistus Sessiooniks 2

- ◆ Sessioon 2: RFM analüüs JAGA-TEE-KOGU-ESITLE meetodil
- ◆ Rollid: A = Data Loading, B = Cleaning, C = RFM Analysis, D = Visualization
- ◆ Oodatav tulemus: terviklik Jupyter Notebook meeskonnalt
- ◆ Marko ootab: VIP-de arv, At-Risk kliendid, soovitusel!
- ◆ Nädal 8 eelvaade: automatiseerimine — Python skript, mis jookseb iga nädal!



Järgmised sammud (jätkub)

- ◆ Sessioon 3 demo: relay-formaat — ajalimiit range, 45 sek per inimene

